

11 февраля 2026 г. Инженерия

Инженерия harness: Codex в мире, ориентированном на агентов

Автор: Райан Лополо, технический персонал

▶ Прослушать статью | 26:22

🔗 Поделиться

За последние пять месяцев наша команда проводила эксперимент: создание и выпуск внутренней бета-версии программного продукта с **0 строк вручную написанного кода**.

У продукта есть внутренние ежедневные пользователи и внешние альфа-тестеры. Он выпускается, разворачивается, ломается и исправляется. Отличие в том, что каждая строка кода — логика приложения, тесты, конфигурация CI, документация, наблюдаемость и внутренние инструменты — была написана Codex. По нашим оценкам, мы создали этот продукт примерно за 1/10 времени, которое потребовалось бы для написания кода вручную.

Люди управляют. Агенты выполняют действия.

Мы намеренно выбрали это ограничение, чтобы создать то, что было необходимо для увеличения скорости разработки на порядок. У нас было несколько недель, чтобы выпустить то, что в итоге оказалось миллионом строк кода. Чтобы сделать это, нам нужно было понять, что меняется, когда основная задача команды разработки ПО меняется с написания кода на проектирование сред, установку намерений и выстраивание контуров обратной связи, которые позволяют агентам Codex выполнять надежную работу.

OpenAI

командой агентов — что сломалось, что усугубилось и как максимально эффективно использовать наш единственный по-настоящему дефицитный ресурс: человеческое время и внимание.

Мы начали с пустого git-репозитория

Первый коммит в пустой репозиторий был сделан в конце августа 2025 года.

Начальная форма — структура репозитория, конфигурация CI, правила форматирования, настройка менеджера пакетов и фреймворк приложения — была сгенерирована Codex CLI с использованием GPT-5 и небольшого набора существующих шаблонов. Даже исходный файл AGENTS.md, который указывает агентам, как работать в репозитории, был написан самим Codex.

Не было заранее написанного человеком кода, на который система могла бы опираться. С самого начала репозиторий был сформирован агентом.

Через пять месяцев репозиторий уже содержал около миллиона строк кода, охватывающих логику приложения, инфраструктуру, инструменты, документацию и внутренние утилиты для разработчиков. За этот период было открыто и объединено примерно 1 500 pull request, при этом Codex развивала небольшая команда всего из трёх инженеров. Это соответствует средней пропускной способности 3,5 PR на инженера в день; что удивительно, пропускная способность *повысилась* по мере того, как команда выросла и теперь насчитывает семь инженеров. Важно отметить, что это не был результат ради результата: продукт использовался сотнями пользователей внутри компании, в том числе активно и ежедневно.

На протяжении всего процесса разработки люди никогда напрямую не писали код. Это стало основной философией команды: **никакого кода, написанного вручную.**

Переосмысление роли разработчика

OpenAI

тип инженерной работы, сосредоточенной на системах, инфраструктуре и оптимизации.

На ранних этапах прогресс был медленнее, чем мы ожидали: не потому, что Codex был неспособен, а потому, что среда была недооценена. Агенту не хватало инструментов, абстракций и внутренней структуры, необходимых для достижения прогресса на пути к высокоуровневым целям. Основная задача нашей команды заключалась в том, чтобы дать агентам возможность выполнять полезную работу.

На практике это означало работу по принципу глубинного поиска: разбиение более крупных целей на меньшие блоки (дизайн, код, проверка, тестирование и т.д.), побуждение агента собирать эти блоки и использование их для разблокировки более сложных задач. Когда что-то не удавалось, решение почти никогда не заключалось в том, чтобы «стараться усерднее». Поскольку единственным способом добиться прогресса было заставить Codex выполнять работу, инженеры всегда брались за задачу и спрашивали: «какой возможности не хватает и как сделать её одновременно понятной и обязательной для агента?»

Люди взаимодействуют с системой почти исключительно через промпты: разработчик описывает задачу, запускает агента и позволяет ему открыть pull request. Чтобы довести PR до завершения, мы инструктируем Codex локально проверять свои изменения, запрашивать дополнительные конкретные проверки у агентов как локально, так и в облаке, отвечать на любые отзывы, предоставленные человеком или агентом, и повторять этот цикл до тех пор, пока все агенты-рецензенты не будут удовлетворены (по сути, это цикл Ральфа Виггума). Codex использует наши стандартные инструменты разработки напрямую (gh, локальные скрипты и навыки, встроенные в репозиторий), чтобы собирать контекст без необходимости копирования и вставки данных в CLI вручную.

Люди могут просматривать pull request'ы, но не обязаны этого делать. Со временем мы направили почти все усилия на то, чтобы проверка осуществлялась агентами напрямую.

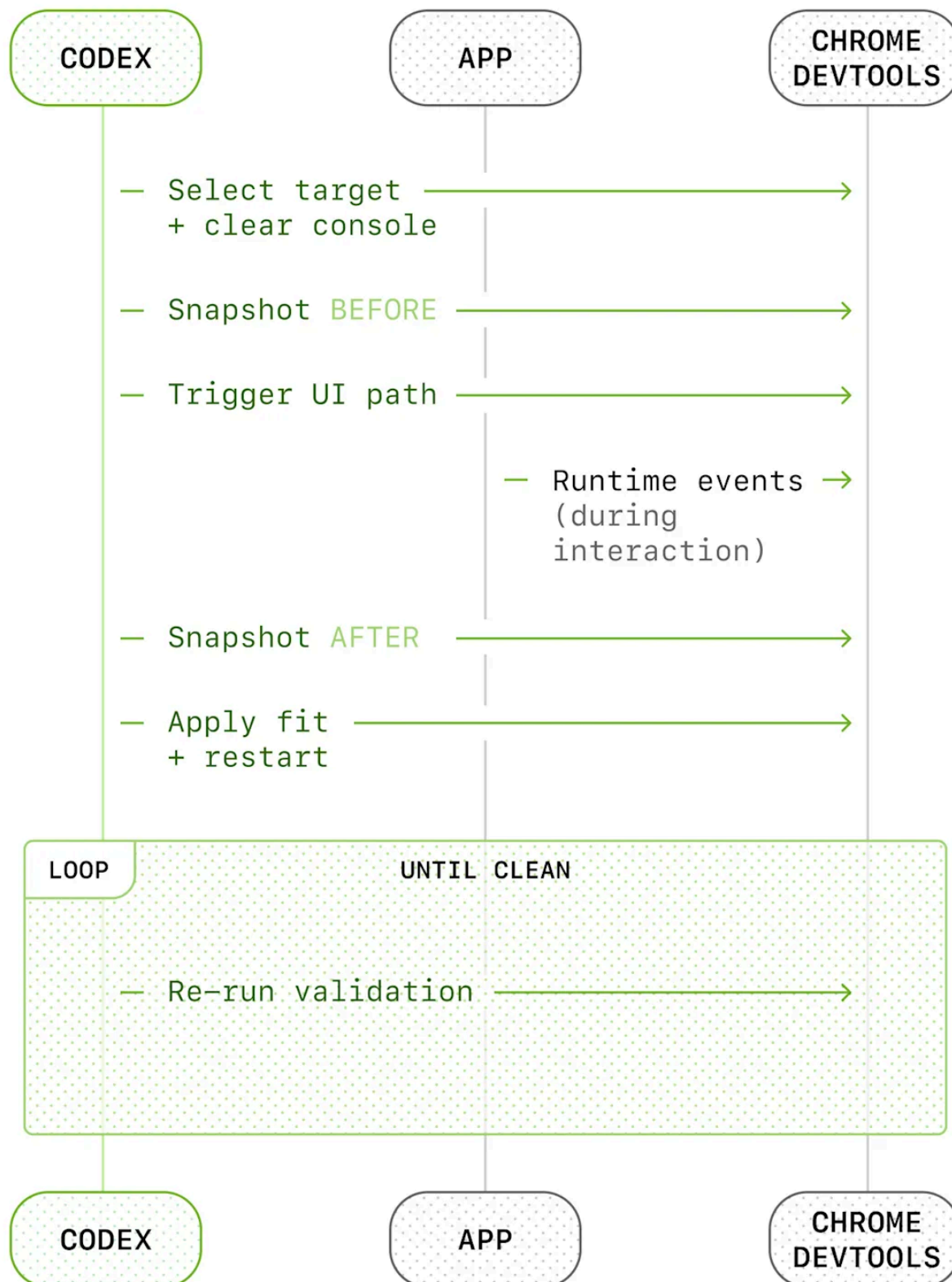
Повышение читаемости приложения

OpenAI

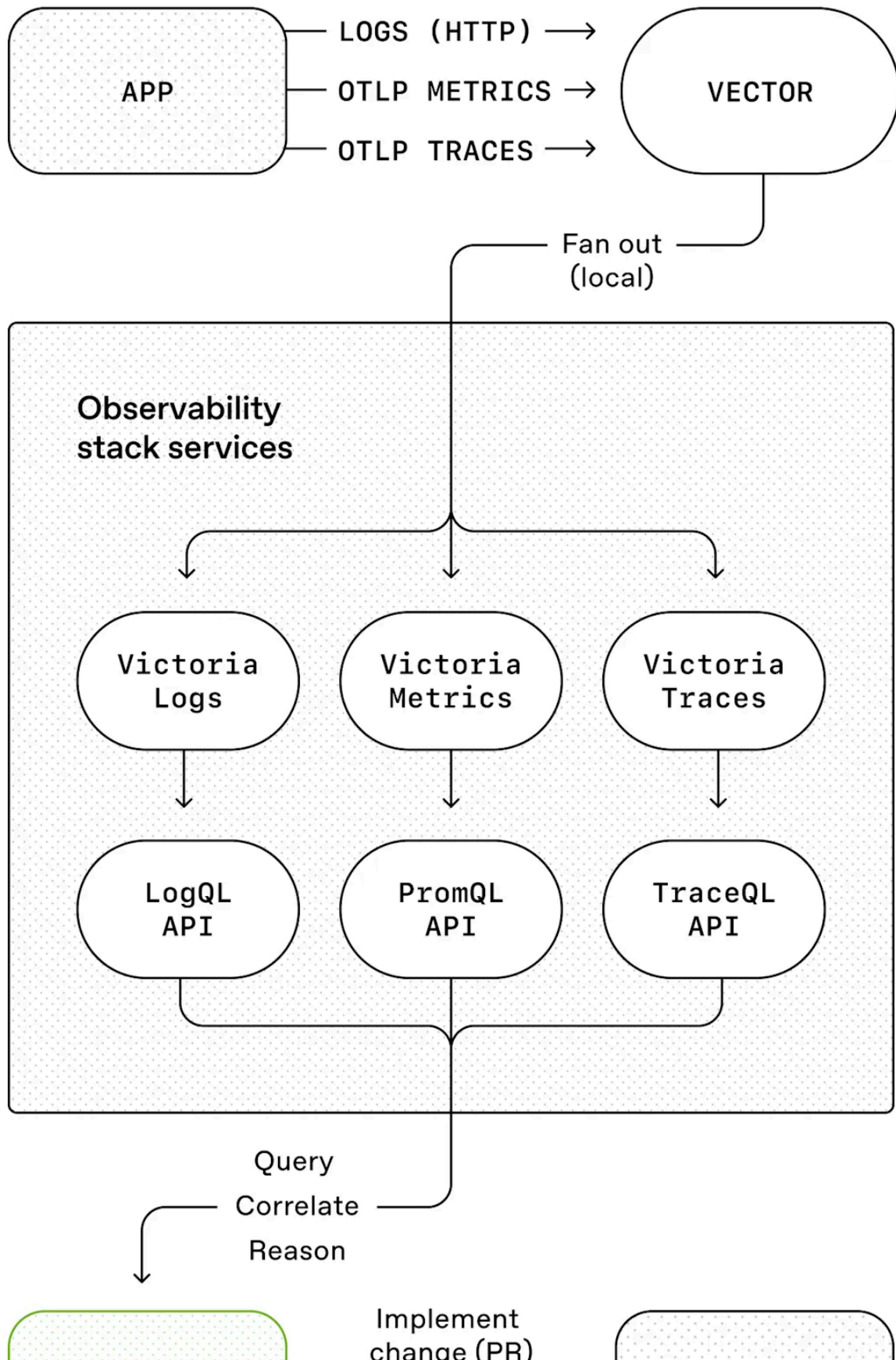
пропускная способность человеческого QA. Поскольку фиксированным ограничением были человеческое время и внимание, мы работали над тем, чтобы добавить агенту больше возможностей, сделав такие элементы, как пользовательский интерфейс приложения, логи и метрики приложения, непосредственно понятными для Codex.

Например, мы сделали приложение загружаемым для каждого git worktree, чтобы Codex мог запускать и управлять одним экземпляром на каждое изменение. Мы также интегрировали протокол Chrome DevTools в среду выполнения агента и разработали навыки для работы с DOM-снимками, скриншотами и навигацией. Это позволило Codex воспроизводить ошибки, проверять исправления и непосредственно анализировать поведение пользовательского интерфейса.

Codex drives the app with Chrome DevTools MCP to validate its work



Giving Codex a full observability stack



OpenAI



Сперва мы опробовали вариант «один крупный файл `AGENTS.md`». Это не сработало, причем по довольно предсказуемым причинам:

- **Контекст — дефицитный ресурс.** Огромный файл с инструкциями вытесняет задачу, код и соответствующую документацию, поэтому агент либо упускает ключевые ограничения, либо начинает оптимизировать неправильные.
- **Слишком масштабное руководство = бесполезное руководство.** Когда «важным» считается всё, концепция «важности» теряется. Агенты в конечном итоге начинают локально сопоставлять данные по шаблонам, вместо того, чтобы выстраивать навигацию.
- **Со временем руководство начинает рушиться.** Монолитное руководство превращается в кладбище устаревших правил. Агенты не могут определить, что в руководстве остается актуальным, люди перестают его поддерживать, и руководство становится попросту бесполезным.
- **Руководство трудно проверить.** Один огромный файл сложно невозможно подвергнуть механическим проверкам (на покрытие, актуальность, владение, перекрестные связи), и отклонение неизбежно.

Итак, вместо того, чтобы относиться к `AGENTS.md` как к энциклопедии, мы относимся к нему как к **оглавлению**.

База знаний репозитория находится в структурированном каталоге `docs/`, который рассматривается как система записи. Краткий `AGENTS.md` (около 100 строк)

Простой Текст



```

1  AGENTS.md
2  ARCHITECTURE.md
3  docs/
4  |— design-docs/
5  |   |— index.md
6  |   |— core-beliefs.md
7  |   └─ ...
8  |— exec-plans/
9  |   |— active/
10 |   |— completed/
11 |   └─ tech-debt-tracker.md
12 |— generated/
13 |   └─ db-schema.md
14 |— product-specs/
15 |   |— index.md
16 |   |— new-user-onboarding.md
17 |   └─ ...
18 |— references/
19 |   |— design-system-reference-llms.txt
20 |   |— nixpacks-llms.txt
21 |   |— uv-llms.txt
22 |   └─ ...
23 └─ NESTING.md

```

Структура хранилища знаний в репозитории.

Проектная документация каталогизируется и индексируется, включая статус проверки и набор основных убеждений, определяющих принципы работы, ориентированные на агента. Документация по архитектуре предоставляет карту доменов верхнего уровня и структуру слоев пакетов. Документ по качеству оценивает каждый домен продукта и архитектурный слой, отслеживая пробелы с течением времени.

Планы рассматриваются как полноценные артефакты. Эфемерные облегченные планы используются для небольших изменений, в то время как сложная работа фиксируется в планах выполнения с журналами прогресса и решений, которые заносятся в репозиторий. Активные планы, завершённые планы и известные технические недоработки — для всего этого устанавливается версия, и все это

OpenAI

Это обеспечивает **постепенное раскрытие информации**: агенты начинают с небольшой, стабильной точки входа, и их обучают, куда смотреть дальше, вместо того чтобы сразу перегружать их.

Мы обеспечиваем это механически. Выделенные линтеры и задания CI проверяют, что база знаний актуальна, содержит перекрёстные ссылки и правильно структурирована. Периодический агент «doc-gardening» сканирует устаревшую или неактуальную документацию, которая не отражает реальное поведение кода, и открывает pull request для внесения исправлений.

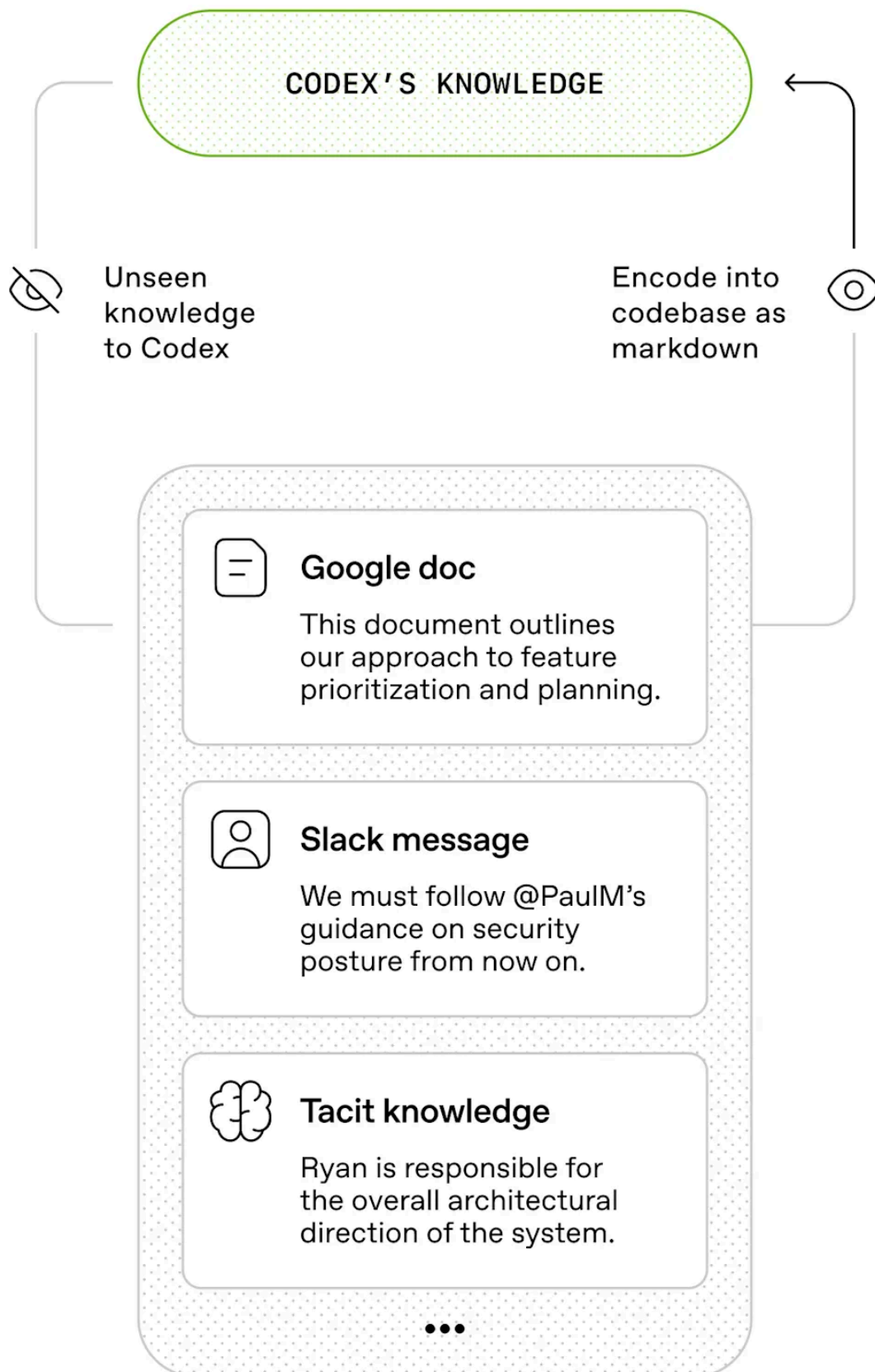
Цель — читаемость для агента

По мере развития кодовой базы должна была развиваться и структура Codex для принятия проектных решений.

Поскольку репозиторий полностью создан агентом, он оптимизирован в первую очередь для *читабельности Codex*. Точно так же, как команды стремятся улучшить навигацию по своему коду для новых инженеров, наша цель как инженеров-людей заключалась в том, чтобы дать агенту возможность рассуждать обо всей предметной области **непосредственно из самого репозитория**.

С точки зрения агента, всё, к чему он не может получить доступ в контексте во время выполнения, фактически не существует. Знания, которые находятся в Google Docs, в чатах или в головах людей, недоступны системе. Локальные для репозитория версионизируемые артефакты (например, код, разметка, схемы, исполняемые планы) — это всё, что он может видеть.

The limits of agent knowledge: What Codex can't see doesn't exist



OpenAI

и больше контекста. Скажем, в Slack прошло обсуждение, которое помогло команде согласовать паттерн архитектуры. Если агент не может его обнаружить, это обсуждение будет для него примерно так же бесполезно, как для нового сотрудника, который придет в компанию через три месяца.

Предоставление Codex большего контекста означает организацию и раскрытие нужной информации, чтобы агент мог рассуждать на её основе, а не перегружать его разрозненными инструкциями. Точно так же, как вы вводите в курс дела нового члена команды по принципам продукта, инженерным нормам и культуре команды (включая предпочтения по эмодзи), предоставление агенту этой информации приводит к более согласованному результату.

Эта формулировка прояснила многие компромиссы. Мы отдавали предпочтение зависимостям и абстракциям, которые можно было полностью интегрировать и осмыслить в репозитории. Технологии, которые часто называют «скучными», как правило, легче моделировать агентам благодаря композиционности, стабильности API и представленности в обучающем наборе. В некоторых случаях было дешевле поручить агенту заново реализовать подмножества функциональности, чем обходить неочевидное поведение вышестоящих компонентов из публичных библиотек. Например, вместо того чтобы подключать универсальный пакет в стиле `p-limit`, мы реализовали собственный вспомогательный метод `map-with-concurrency`: он тесно интегрирован с нашей системой `OpenTelemetry`, имеет 100% покрытие тестами и ведёт себя ровно так, как ожидается для нашей среды выполнения.

Перевод системы в форму, которую агент может проверять, валидировать и изменять напрямую, увеличивает рычаг воздействия — не только для Codex, но и для других агентов (например, [Aardvark](#)), которые также работают над кодовой базой.

Поддержание архитектуры и соблюдение предпочтений

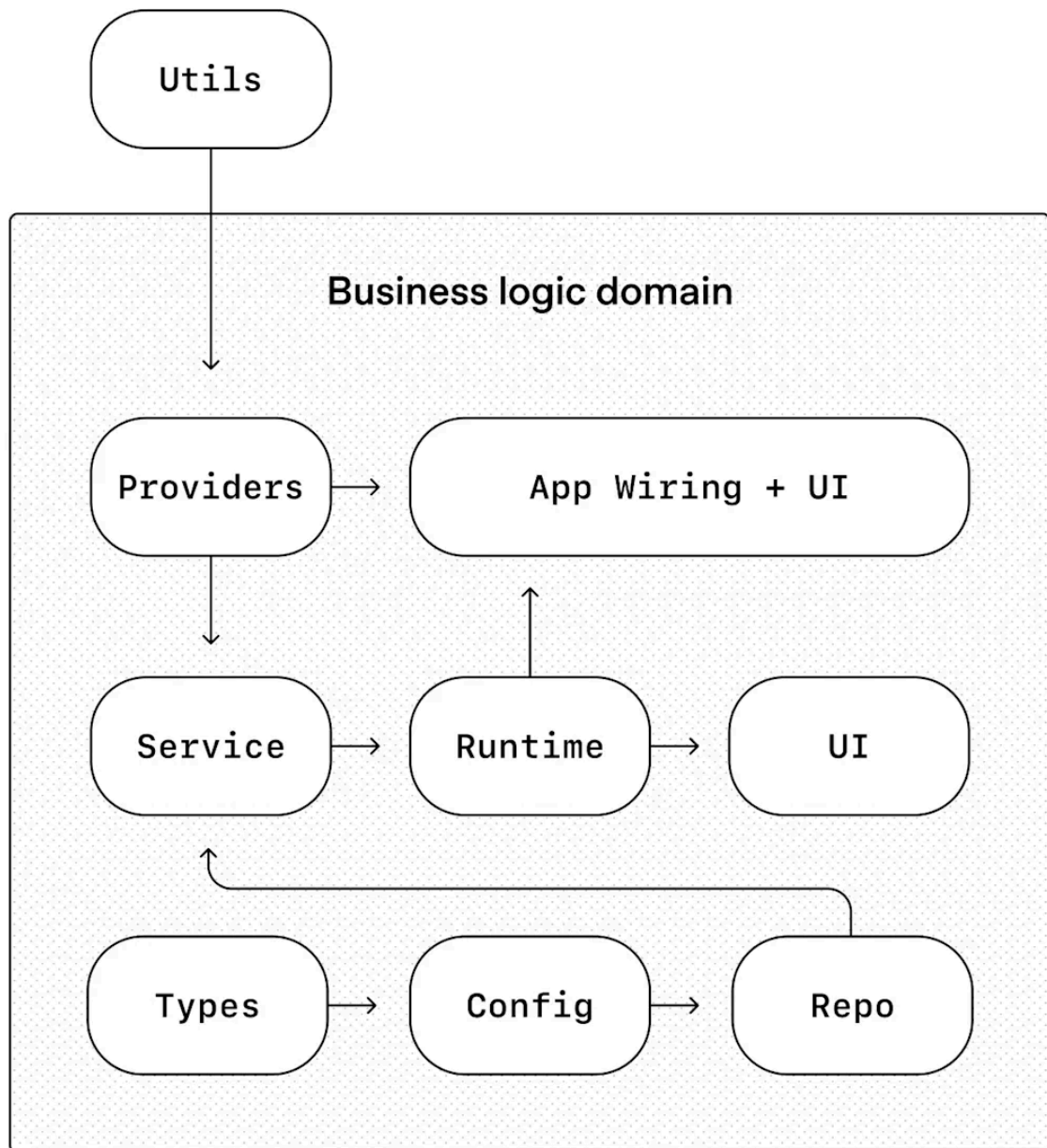
OpenAI

сгенерированной агентом кодовой базы. **Обеспечивая соблюдение инвариантов, а не контролируя каждую деталь реализации, мы позволяем агентам быстро выпускать изменения, не подрывая основу.** Например, мы требуем, чтобы Codex разбирал формы данных на границе, но не предписываем, как именно это должно происходить (модели, похоже, нравится Zod, но мы не указывали эту конкретную библиотеку).

Агенты наиболее эффективны в средах со строгими границами и предсказуемой структурой, поэтому мы разработали приложение на основе жесткой архитектурной модели. Каждая предметная область разделена на фиксированный набор слоев со строго проверенными направлениями зависимостей и ограниченным набором допустимых связей. Эти ограничения механически применяются с помощью пользовательских линтеров (разумеется, сгенерированных Codex!) и структурных тестов.

Схема ниже показывает правило: в рамках каждого предметного домена (например, Настройки приложения), код может показывать только последовательную зависимость через фиксированный набор слоёв (Types → Config → Repo → Service → Runtime → UI). Сквозные аспекты (аутентификация, коннекторы, телеметрия, флаги функций) входят через единый явный интерфейс: Providers. Все остальное запрещено и контролируется механически.

Layered domain architecture with explicit cross-cutting boundaries



Это тот тип архитектуры, который обычно откладывают до тех пор, пока у вас не наберется сотня инженеров. С агентами программирования это ранее

На практике мы обеспечиваем соблюдение этих правил с помощью пользовательских линтеров и структурных тестов, а также небольшого набора «предпочтительных инвариантов». Например, мы статически обеспечиваем соблюдение структурированного логирования, соглашений об именовании для схем и типов, ограничений на размер файлов и специфичных для платформы требований к надежности с помощью пользовательских линтеров. Поскольку линтеры являются настраиваемыми, мы формулируем сообщения об ошибках так, чтобы внедрять инструкции по устранению неполадок в контекст агента.

В рабочем процессе, ориентированном на человека, эти правила могут казаться педантичными или ограничивающими. С агентами они становятся множителями: после внедрения они применяются везде одновременно.

В то же время мы четко указываем, где ограничения имеют значение, а где — нет. Это похоже на руководство крупной инженерной платформенной организацией: централизованно устанавливать границы, предоставлять автономию на местном уровне. Вам глубоко важны границы, корректность и воспроизводимость. В рамках этих границ вы предоставляете командам — или агентам — значительную свободу в том, как формулируются решения.

Полученный код не всегда соответствует человеческим стилистическим предпочтениям, и это нормально. Пока результат корректен, поддерживаем и читаем для будущих запусков агента, он соответствует требованиям.

Человеческие предпочтения непрерывно передаются обратно в систему. Комментарии к проверке, рефакторинг pull request'ов и ошибки, видимые пользователям, фиксируются как обновления документации или кодируются непосредственно в инструменты. Когда документации не хватает, мы внедряем правило в код.

Пропускная способность меняет философию слияния

OpenAI

инженерные нормы стали контрпродуктивными.

Репозиторий работает с минимальным количеством блокирующих merge-гейтов. Pull request'ы недолговечны. Нестабильные тесты часто устраняются повторными прогонами, а не блокируют прогресс на неопределённый срок. В системе, где пропускная способность агента значительно превышает возможности человеческого внимания, исправления обходятся дешево, а ожидание — дорого.

Это было бы безответственно в среде с низкой пропускной способностью. Здесь это часто является правильным компромиссом.

Что на самом деле означает «созданный агентом»

Когда мы говорим, что кодовая база создана агентами Codex, мы имеем в виду всю кодовую базу.

Агенты создают:

- Код продукта и тесты
- Настройку CI и инструменты для релиза
- Внутренние инструменты разработчика
- Историю документации и проектирования
- Системы оценки
- Комментарии к проверке и ответы
- Скрипты, управляющие самим репозиторием
- Файлы определения панели управления для продакшена

Люди всегда остаются в процессе, но работают на другом уровне абстракции, нежели раньше. Мы расставляем приоритеты в работе, преобразуем отзывы пользователей в критерии приемлемости и проверяем результаты. Когда агент испытывает трудности, мы воспринимаем это как сигнал: определяем, чего не

OpenAI

собирают отзывы, отвечают в обсуждениях, отправляют обновления и часто объединяют свои собственные pull request'ы.

Повышение уровней автономности

По мере того как всё больше элементов цикла разработки кодировалось непосредственно в систему — тестирование, валидация, проверка, обработка обратной связи и восстановление — репозиторий недавно преодолел важный порог, при котором Codex может полностью управлять созданием новой функции.

Получив один промпт, агент теперь может:

- Проверить текущее состояние кодовой базы
- Воспроизвести зафиксированную ошибку
- Записать видеоподтверждение ошибки
- Реализовать исправление
- Проверить исправление, запустив приложение
- Записать второе видео с подтверждением решения проблемы
- Открыть pull request
- Ответить на отзыв от агента и от человека
- Выявить и устранить проблемы в сборке
- Передать проблему на рассмотрение человеку — только в тех случаях, когда требуется человеческое суждение
- Объединить изменения

Это поведение во многом зависит от конкретной структуры и инструментов этого репозитория и не должно считаться обобщаемым без сопоставимых вложений — по крайней мере, пока что.

Полная автономия агента создает и новые проблемы. Codex воспроизводит шаблоны, которые уже существуют в репозитории — даже неравномерные или неоптимальные. Со временем это неизбежно приводит к отклонениям.

Изначально люди решали эту задачу вручную. Раньше наша команда тратила каждую пятницу (20% недели) на уборку «ИИ-мусора». Неудивительно, что это не прижилось.

Вместо этого мы начали встраивать так называемые «золотые принципы» прямо в репозиторий и запустили регулярный процесс очистки. Это механические правила с чёткой позицией, которые помогают поддерживать кодовую базу читаемой и единообразной для будущих запусков агента. Например: (1) мы предпочитаем общие пакеты утилит вместо самописных вспомогательных модулей, чтобы инварианты оставались в одном месте; (2) мы не «прощупываем» данные в стиле YOLO — проверяем границы или полагаемся на SDK с типами, чтобы агент случайно не строил логику на предположениях о структуре данных. Регулярно у нас запускается набор фоновых задач Codex, которые отслеживают отклонения, обновляют оценки качества и открывают точечные пул-реквесты на рефакторинг. Большинство из них можно просмотреть менее чем за минуту и объединить автоматически.

Это работает как сбор мусора. Технические недоработки — это как кредит с высокой процентной ставкой: почти всегда лучше погашать его постепенно и небольшими частями, чем позволять долгу накапливаться и разбираться с ним болезненными рывками. Предпочтения человека фиксируются один раз, а затем непрерывно применяются к каждой строке кода. Это также позволяет нам ежедневно выявлять и устранять неприемлемые шаблоны, вместо того чтобы позволять им распространяться в кодовой базе в течение нескольких дней или недель.

Чему мы всё ещё учимся

Эта стратегия до сих пор хорошо работала вплоть до внутреннего запуска и

долгосрочной поддержке.

Чего мы пока не знаем, так это того, как архитектурная целостность развивается на протяжении многих лет в полностью сгенерированной агентами системе. Мы все еще разбираемся с тем, где человеческое суждение дает наибольший эффект, и как формализовать это суждение так, чтобы оно накапливалось. Кроме того, мы еще не знаем, как эта система будет развиваться по мере роста продуктивности моделей со временем.

Что стало очевидным: разработка программного обеспечения по-прежнему требует дисциплины, но эта дисциплина проявляется больше в структуре, чем в коде. Инструменты, абстракции и циклы обратной связи, которые поддерживают связность кодовой базы, становятся все более важными.

Наши самые сложные задачи сейчас сосредоточены на проектировании сред, контуров обратной связи и систем управления, которые помогают агентам достигать нашей цели: создавать и поддерживать сложное, надежное программное обеспечение в масштабах.

По мере того как такие агенты, как Codex, берут на себя всё более значительные части жизненного цикла программного обеспечения, эти вопросы будут становиться ещё более актуальными. Мы надеемся, что, поделившись некоторыми ранними уроками, мы поможем вам понять, куда стоит вложить усилия, чтобы сосредоточиться на создании того, что просто работает.

ОТЗЫВ О ПЕРЕВОДЕ

Насколько легко читается эта страница на русском?

☒ Отлично

☐ Хорошо

Codex 2026

Автор

Ryan Lopopolo

Благодарности

Особая благодарность Виктору Чжу и Заку Броку, которые внесли вклад в этот пост, а также всей команде, создавшей этот новый продукт.

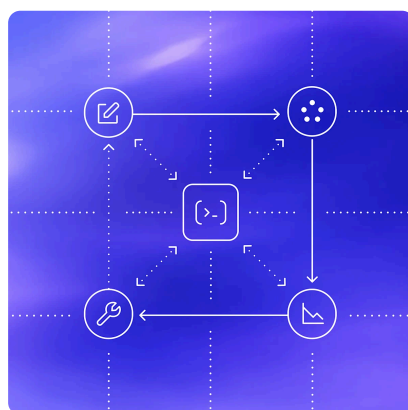
Читать далее

Просмотреть все



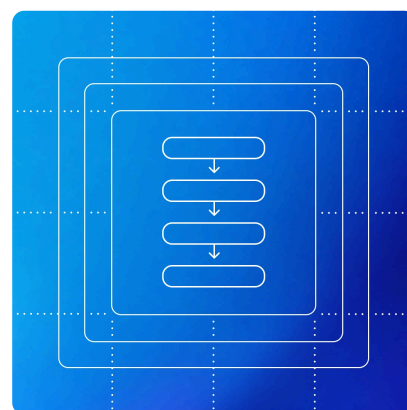
Core dump
epidemiology: fixing
an 18-year-old bug

Инженерия 30 июн. 2026 г.



Создание
самообучающихся
налоговых агентов
с Codex

Инженерия 27 мая 2026 г.



Создание
безопасной и
эффективной
песочницы для
работы Codex в
Windows

Инженерия 13 мая 2026 г.

Исследования	Продукты	Business	Компания	Подробнее
Каталог исследований	ChatGPT ↗	Обзор	О нас	Истории
Обзор исследований	ChatGPT Business ↗	Решения	Наш устав	Академия
Экономические исследования	ChatGPT Enterprise ↗	Ресурсы	Карьеры	Supply Co.
Последние достижения	ChatGPT для Образования ↗	Истории наших клиентов	Новости	Прямые трансляции
GPT-5.6	Codex	Партнерская сеть	Поддержка	Подкаст
GPT-5.5	Примечания к релизу	Связаться с отделом продаж	Справочный центр ↗	RSS
GPT-5.4	Платформа API	Разработчики		Условия и правила использования
Безопасность	Обзор	Apps SDK ↗		Условия использования
Подход к безопасности	Вход в API ↗	Открытые модели		Политика конфиденциальности
Безопасность развертывания ↗	Документы ↗	Документы ↗		Другие политики
Безопасность и конфиденциальность		Ресурсы ↗		
Доверие и прозрачность		Форум разработчиков ↗		